

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТСиВС)

Расчетно-графическое задание
по дисциплине
«WEB-Технологии»

по теме:
СОЗДАНИЕ ТЕСТОВОЙ ВИДЕОПЛАТФОРМЫ С ИСПОЛЬЗОВАНИЕМ
ТЕХНОЛОГИЙ PYTHON ДЛЯ БЭКЕНДА, REACT JS ДЛЯ ФРОНТЕНДА
И FIGMA ДЛЯ ДИЗАЙНА

Студент:
Группа № ИКС-532

Лёшин А. С.

Преподаватель:
Старший преподаватель

Андреев А.В.

Новосибирск 2026 г.

1 Введение

Проект представляет собой современное веб-приложение с распределенной архитектурой клиент-сервер, использующее фреймворк Django для реализации серверного бэкенда и библиотеку React для построения интерактивного фронтенда.

1.1 Задачи проекта

- Разработка структурированного и документированного REST API на базе Django REST Framework.
- Создание надежной системы регистрации и авторизации пользователей с использованием безопасных JWT-токенов.
- Реализация серверного и клиентского функционала для загрузки, обработки и потокового воспроизведения медиафайлов.
- Разработка отзывчивого пользовательского интерфейса на React с применением утилитарного CSS-фреймворка Tailwind CSS.
- Конфигурация серверного окружения и деплой готового проекта на удаленный VDS сервер.

2 Архитектура проекта

2.1 Стек технологий

Компоненты разработанной системы и их технологическое назначение подробно систематизированы и представлены в таблице 1.

Таблица 1 — Технологическая спецификация компонентов

Компонент системы	Используемая технология
Бэкенд-платформа	Django 5.0, Django REST Framework
Аутентификация сессий	JWT (djangorestframework-simplejwt)
База данных	Реляционная СУБД SQLite 3
Фронтенд-архитектура	React 18, React Router DOM v6
Стилизация интерфейса	Tailwind CSS
Асинхронный транспорт	HTTP-клиент Axios
Веб-сервер удаленного деплоя	Nginx (Amvera Cloud / VDS)

2.2 Структура проекта

Разделение кодовой базы на независимые каталоги бэкенда и фронтенда обеспечивает изолированную разработку каждого слоя системы. Логическое дерево каталогов проекта приведено в листинге 1.

Листинг 1 — Структура проекта MyTube

```
1 video-platform/
2   backend/           # Django бэкенд
3     core/            # Настройки проекта
4     users/           # Приложение управления пользователями
5     videos/          # Приложение управления видеоконтентом
6     media/           # Хранилище загруженных файлов
7   frontend/         # React фронтенд
8     public/          # Статические файлы
9     src/
10       components/   # Переиспользуемые React компоненты
11       pages/        # Страницы приложения
12       contexts/     # Глобальные стейты (Context API)
13       App.js        # Главный корневой компонент
```

3 Разработка бэкенда

3.1 Модели данных

Архитектура базы данных построена на расширении стандартной модели пользователя для кастомизации профилей (листинг 2) и сущности хранения метаданных видеоконтента (листинг 3).

Листинг 2 — Модель пользователя User

```
1 class User(AbstractUser):
2     avatar = models.ImageField(upload_to='avatars/', null=True,
3                                blank=True)
4     banner = models.ImageField(upload_to='banners/', null=True,
5                                blank=True)
6     bio = models.TextField(max_length=500, blank=True, default='')
7     location = models.CharField(max_length=100, blank=True,
8                                 default='')
9
```

```

6     website = models.URLField(blank=True, default='')
7     birth_date = models.DateField(null=True, blank=True)
8     subscribers = models.ManyToManyField('self', symmetrical=
        False, blank=True)

```

Листинг 3 — Модель видео Video

```

1 class Video(models.Model):
2     title = models.CharField(max_length=200)
3     description = models.TextField(blank=True)
4     video_file = models.FileField(upload_to='videos/%Y/%m/')
5     thumbnail = models.ImageField(upload_to='thumbnails/%Y/%m/',
        null=True, blank=True)
6     author = models.ForeignKey(User, on_delete=models.CASCADE,
        related_name='videos')
7     views = models.IntegerField(default=0)
8     likes = models.ManyToManyField(User, related_name='
        liked_videos', blank=True)
9     created_at = models.DateTimeField(auto_now_add=True)

```

3.2 API Эндпоинты

Маршрутизация запросов к REST API серверной части представлена в таблице 2.

Таблица 2 — Маршруты REST API подсистемы

Метод	Эндпоинт	Описание функционала
POST	/api/users/register/	Регистрация нового аккаунта
POST	/api/users/login/	Вход в систему (генерация JWT токена)
GET	/api/users/me/	Получение данных текущей сессии
GET	/api/videos/	Получение полного списка видеороликов
GET	/api/videos/recommended/	Запрос блока рекомендованных видео
POST	/api/videos/	Загрузка нового видео на сервер
POST	/api/videos/{id}/like/	Переключение лайка на контенте
POST	/api/videos/{id}/view/	Инкремент счетчика просмотров
GET	/api/videos/{id}/comments/	Запрос комментариев под видео
POST	/api/videos/{id}/comments/	Добавление текстового комментария
DELETE	/api/videos/{id}/	Изолированное удаление видеоролика
PATCH	/api/users/{id}/update-profile/	Валидация и обновление профиля

3.3 JWT Аутентификация

Интеграция авторизационных ограничений в настройки проекта Django представлена в листинге 4.

Листинг 4 — Настройка JWT в файле settings.py

```
1 REST_FRAMEWORK = {
2     'DEFAULT_AUTHENTICATION_CLASSES': (
3         'rest_framework_simplejwt.authentication.
4           JWTAuthentication',
5     ),
6 }
7 SIMPLE_JWT = {
8     'ACCESS_TOKEN_LIFETIME': timedelta(days=1),
9     'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
10 }
```

4 Разработка фронтенда

4.1 Основные компоненты

4.1.1 VideoCard — карточка отображения видео

Компонент отвечает за рендеринг превью, заголовка и автора контента в общей сетке выдачи (листинг 5).

Листинг 5 — Компонент карточки видео VideoCard.jsx

```
1 export default function VideoCard({ video }) {
2     return (
3         <Link to={`\video/${video.id}`} className="block group">
4             <div className="bg-gray-900 rounded-xl overflow-hidden">
5                 <div className="relative pb-[56.25%]">
6                     {video.thumbnail ? (
7                         <img src={video.thumbnail} alt={video.title}
8                           className="absolute inset-0 w-full h-full object-
9                             cover" />
10                    ) : (
11                        <div className="absolute inset-0 bg-gradient-to-br
12                          from-purple-900 to-pink-900" />
13                    )
14                }
15            </div>
16        </Link>
17    )
18 }
```

```

10     })
11   </div>
12   <div className="p-3">
13     <h3 className="font-semibold text-white">{video.title}
14     </h3>
15     <p className="text-sm text-gray-400">{video.author?.
16       username}</p>
17   </div>
18 </div>
19 </Link>
20 );
21 }

```

4.1.2 Механизм динамического поиска

Запросы к API-серверу для фильтрации контента по строковым паттернам без перезагрузки страницы представлены в листинге 6.

Листинг 6 — Реализация поисковой фильтрации

```

1 const performSearch = async (query) => {
2   const response = await axios.get(`/videos/?search=${
3     encodeURIComponent(query)}`);
4   setVideos(response.data.results || response.data);
5 };
6
7 useEffect(() => {
8   const params = new URLSearchParams(window.location.search);
9   const query = params.get('search');
10  if (query) performSearch(query);
11 }, [window.location.search]);

```

4.2 AuthContext — управление глобальным стейтом авторизации

Использование React Context API для обеспечения сквозной аутентификации сессий приведено в листинге 7.

Листинг 7 — Контекст авторизации AuthContext.jsx

```

1 const AuthContext = createContext();
2
3 export const AuthProvider = ({ children }) => {

```

```

4  const [user, setUser] = useState(null);
5
6  const login = async (username, password) => {
7      const response = await axios.post('/users/login/', { username
8          , password });
9      localStorage.setItem('access_token', response.data.access);
10     setUser(response.data.user);
11     return true;
12 };
13
14 return (
15     <AuthContext.Provider value={{ user, login, logout }}>
16         {children}
17     </AuthContext.Provider>
18 );

```

5 Деплой на сервер

5.1 Настройка системного окружения

Установка необходимого серверного программного обеспечения на Linux-дистрибутив VDS (листинг 8).

Листинг 8 — Установка зависимостей на сервере

```

1 apt update && apt upgrade -y
2 apt install -y python3-pip python3-venv nginx nodejs npm

```

5.2 Запуск бэкенда через Gunicorn

Создание изолированного окружения и инициализация многопоточного WSGI-сервера (листинг 9).

Листинг 9 — Настройка и старт процессов Gunicorn

```

1 cd /root/video-platform/backend
2 python3 -m venv venv
3 source venv/bin/activate
4 pip install -r requirements.txt
5 gunicorn --workers 3 --bind 0.0.0.0:8000 backend.wsgi:application

```

5.3 Конфигурация веб-сервера Nginx

Настройка проксирования входящих запросов к API и раздача статического билда фронтенда (листинг 10).

Листинг 10 — Конфигурационный файл Nginx

```
1 server {
2     listen 80;
3     server_name 62.109.1.29;
4
5     location /api/ {
6         proxy_pass http://127.0.0.1:8000/api/;
7     }
8
9     location / {
10        root /root/video-platform/frontend/build;
11        try_files $uri $uri/ /index.html;
12    }
13 }
```

7 Результаты работы

В ходе выполнения проекта были достигнуты следующие практические результаты:

- Разработана и отлажена безопасная регистрация и авторизация пользователей с JWT-токенами.
- Реализован плеер для плавного просмотра видеоконтента.
- Реализован асинхронный модуль загрузки видеоматериалов с генерацией уникальных превью.
- Интегрирована интерактивная система лайков и текстовых комментариев.
- Разработан модуль подписки на каналы выбранных авторов.
- Внедрен реактивный поиск по названиям видеороликов на главном экране.
- Обеспечено редактирование профилей (кастомизация баннера, аватара, биографии).

8 Выводы

В ходе выполнения расчетно-графического задания была успешно спроектирована и программно реализована полноценная видеоплатформа «MyTube». Разработано отказоустойчивое REST API средствами Django REST Framework с JWT-аутентификацией сессий. Создан интерактивный визуальный фронтенд-компонент на React с применением утилитарной стилизации Tailwind CSS. Все ключевые модули, включая подписки, комментарии, динамические счетчики и загрузку мультимедиа, были успешно интегрированы и развернуты на удаленном сервере VDS. Поставленные задачи выполнены в полном объеме, закреплен практический опыт построения современных клиент-серверных приложений.

9 Ссылки на ресурсы

– **GitHub репозиторий проекта:**

<https://github.com/ArtemLeshin/video-hosting>

– **Адрес опубликованного веб-интерфейса (Netlify):**

<https://exquisite-dusk-620e3d.netlify.app>

– **Базовый адрес бэкенд-сервера API (Amvera):**

<https://mutube-dreamshelter.amvera.io>

– **Презентация выполнения работы сайта находится в самом низу файла README.md**

<https://github.com/ArtemLeshin/video-hosting>